

worm_paper_final_newbakiii (1) (1).docx

 Adani University

Document Details

Submission ID

trn:oid:::20705:135500926

Submission Date

Apr 17, 2026, 9:23 AM GMT+5:30

Download Date

Apr 17, 2026, 9:29 AM GMT+5:30

File Name

worm_paper_final_newbakiii (1) (1).docx

File Size

39.5 KB

9 Pages

3,610 Words

20,403 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Worm Attacks in Cybersecurity:

Propagation Mechanisms, Detection Strategies and Mitigation Approaches

Vidhi Doshi, Reeya Patel, Prachi Gohil

Department of CSE(AIML), Faculty of Engineering, Adani University, Ahmedabad, India

Abstract: Computer worms are programs that replicate themselves and spread through computer networks without requiring any host program or user intervention by exploiting vulnerabilities in software. Worms differ from viruses because they operate independently without any user intervention and are highly challenging to control when they spread. The study will examine the structure of worms, their propagation mechanisms such as random, linear, or targeted scanning, and their impact. Theoretical simulations based on the SIR (Susceptible-Infected-Recovered) model are used to compare worm spread in three scenarios: no defense, patching 50% of 2000 random infected nodes, and intrusion detection with patches. The results indicate that rates of infection can be greatly reduced if software is patched early and monitored in real time. It compares the strengths and weaknesses of three types of detection methods — signature, anomaly, and behavior. The paper ends with tangible mitigation strategies to create more resilient network infrastructures.

Keywords—*computer worms, network security, worm propagation, intrusion detection, SIR model.*

1. INTRODUCTION

The concept of worms has remained a long-standing threat in network security, dating back to the late 1980s. Worms are described as a type of replicating malware, which spreads through a network using vulnerabilities, without any requirement for a host file or any interaction with a user.

The economic loss associated with worms has been significant, with the ILOVEYOU worm, which spread in 2000, accounting for a loss of \$10 billion, while Conficker, which spread in 2008, infected millions of machines in government and military networks worldwide. Modern worms are found to carry secondary payloads, which include ransomware, spyware, and botnet software. The mechanisms, propagation, and mitigation of worms are discussed in this work in Sections 2, 3, and 4, respectively.

1.1. General Background and Worm Evolution

The Morris Worm, introduced in 1988, is considered one of the first worms that has been widely documented. The worm targeted Unix-based sendmail and fingerd. Even though the worm was not programmed to cause harm, it consumed enough resources to cause problems on the burgeoning Internet. Since then, worms have shown significant improvements in sophistication, such as polymorphism to evade detection, the use of rootkit technology to remain invisible from the OS, and remote update capabilities. Stuxnet, released in 2010, has shown the potential of worms to be used for cyber warfare, targeting industrial control systems for physical destruction.

Table 1. Comparison of notable computer worms across key parameters

Worm Name	Year	Propagation Method	Target Vulnerability	Estimated Damage
Morris Worm	1988	Sendmail/Fingerd exploit	Unix buffer overflow	~\$10 Million (USD)
Code Red	2001	HTTP buffer overflow	IIS web server flaw	~\$2.6 Billion (USD)
Blaster	2003	RPC DCOM exploit	Windows XP/2000 flaw	~\$320 Million (USD)
ILOVEYOU	2000	Email/VBScript	Windows scripting host	~\$10 Billion (USD)
Conficker	2008	SMB vulnerability/USB	MS08-067 (Windows RPC)	~\$9.1 Billion (USD)

2. RELATED WORK

A considerable body of research exists on computer worm propagation, detection, and mitigation. This section reviews five notable contributions and identifies their limitations to establish the context for the present work.

Zou, Gong, and Towsley (2002) developed one of the earliest mathematical models for worm propagation using differential equations derived from epidemiology. Their work applied the simple epidemic model to Code Red worm data and successfully estimated the basic reproduction number R_0 for internet-scale networks. The model demonstrated how early intervention significantly limits total infection. However, the study assumed a homogeneous, fully-connected network topology, which does not reflect the heterogeneous and scale-free nature of real-world networks. Also, this model did not take into account the fact that the worms have the capability to adapt their behavior and counter any measures taken against them [1]

Moore, Shannon, and Claffy in 2003 explored ways of containing worms spreading through rapid responses using rate-limiting techniques and studied mechanisms involved in edge routers. The experiment proved that a delay of a few seconds on the part of the defenses could result in the infection of several susceptible hosts by the worms. However, its major drawback is that aggressive rate limiting techniques may unintentionally interfere with normal traffic volumes such as database and video transfers.

Porras et al. (2009) – Conficker Working Group undertook an extensive reverse engineering process that revealed the various stages in the command and control infrastructure, DGA, and peer to peer updating process used by the Conficker worm. It provided vital information that enabled a global response to be coordinated involving ICANN and different registrars in order to prevent Conficker from performing its domain lookup. The weakness of this contribution is that this information only became available retrospectively; by then, millions of computers had been compromised. There is also no discussion on how to detect similar DGA worms [3].

In their **2014** publication, **Kaur & Singh** provide a complete benchmark for intrusion detection system (IDS) types, which consists of testing IDS performance when utilizing different IDS detection methods: signature-based detection, anomaly-based detection, and hybrid detection. In particular, they have found that the use of multiple IDS detection methods within a single method, called hybrid detection, is effective, as hybrid detection leads to better IDS detection with lower number of false positives. However, this work uses a 1999 benchmark data set that is no longer adequate as it does not take modern day attacks into account [4].

According to **Manavi & Hamzeh (2017)**, a technique has been introduced for detecting scanning activity using Support Vector Machines (SVM). The system analyzed traffic metadata features — such as

connection rate, port distribution, and packet size variance — without requiring deep packet inspection, making it suitable for encrypted traffic scenarios. In spite of the merits that this technique holds, the drawback associated with this technique is that it demands a huge amount of training data to train the model with, which might not be possible in all cases [5].

One of the first in-depth studies of the Morris Worm and its propagation characteristics was done by **Spafford (1989)**. This study offered information about the possibility of exploiting the security weaknesses of networked applications like sendmail and fingerd to facilitate worm propagation. It also identified the negative effects of unrestricted replication, causing congestion and failure of the networks. Nevertheless, the study was limited to describing the phenomenon and did not propose any generalizable models that would predict the future behavior of worms, especially the large scale and adaptive ones [6].

With the help of **Falliere, Murchu, and Chien (2011)** and their analysis of the Stuxnet worm, there are possible to learn about the sophisticated techniques for spreading of this worm as well as exploits being used as part of Stuxnet. Moreover,

there are possible to understand that Stuxnet has been used to attack a certain target. Therefore, it becomes evident that it can be done through modern worms and what elements are required – stealth, encryption, and targeting, etc. Despite all the great contributions that this paper has made in regards to advanced persistent threats, the problem is that only one worm was analyzed [7].

Gaps in the Literature and Contributions of the Current Paper: The above-cited studies deal with certain aspects of worm propagation and/or detection separately. While previous literature makes use of

either homogeneous networks or outdated data or analyzes past incidents, this paper aims to fill these research gaps by:(a) modeling worm propagation using the SIR approach under the assumption of scale-free network structure;(b) carrying out simulations of worm spread for three possible defense strategies: absence of any defense measures; patching; and worms being detected using IDS techniques; and (c) comparing the different approaches used in detection with practical recommendations regarding multiple layers of security.

Table 2. Summary of Related Work

Author(s)	Year	Contribution	Limitation
Zou et al.	2002	Developed Code Red propagation model using differential equations; quantified R0 in internet-scale networks.	Model assumes homogeneous network topology; real networks have heterogeneous connectivity.
Moore et al.	2003	Analyzed active worm containment via rate-limiting infected hosts; proposed reactive throttling at edge routers.	Rate-limiting can impact legitimate traffic; delayed detection reduces effectiveness.
Symantec / Porras et al.	2009	Reverse-engineered Conficker worm; documented multi-stage C&C architecture and domain generation algorithm.	Analysis was retrospective; real-time detection of such worms remained challenging.
Kaur & Singh	2014	Surveyed signature-based, anomaly-based, and hybrid IDS approaches; benchmarked	KDD Cup dataset is outdated; results may not generalize to modern zero-day exploits.

Author(s)	Year	Contribution	Limitation
		detection accuracy on KDD Cup dataset.	
Manavi & Hamzeh	2017	Proposed behavior-based detection using machine learning (SVM) to identify worm-like scanning patterns in network traffic.	High computational overhead; requires extensive labeled training data not always available.
Spafford	1989	Analysis of the Morris Worm; offered early understanding of worms that exploit vulnerabilities within network services.	Case study on one particular worm from history; fails to provide any models for today's complex networks.
Falliere, Murchu & chien	2011	Analysis of Stuxnet; identified sophisticated techniques used by Stuxnet for its propagation and attacks.	Narrow scope of application; only applies to Stuxnet or worms similar to it.

3. MATERIALS AND METHOD

3.1. Structure of a Computer Worm

Typically, a traditional computer worm includes four major components:

- Target finding and scanning – this identifies the possible hosts for the worm;
- Propagation – this duplicates the worm to the identified hosts;
- Payload – this is the primary malicious activity of the worm;
- Command and control – this is a typical feature of modern worms that allows the worm to receive commands from a remote human operator.

The scanning process is the most important factor in controlling the speed of the worm. The most common scanning methods are:

- Random Scanning – sending random IP addresses (high volume of traffic and a slow initial phase);
- Sequential Scanning – scanning a specified IP address range (discernible);

- Hit-list Scanning – scanning a pre-loaded list of vulnerable hosts (fast initial phase);
- Topological Scanning – the worm uses data gathered from the compromised host (e.g., routing tables and browser history).

3.2. Propagation Techniques and Payload Execution

Worm propagation is carried out through software vulnerabilities. Emphasis is given to buffer overflow attacks, which take place when the data received is larger than the allocated space in the assigned buffer. Morris, Blaster, and Code Red worms were propagated using this type of exploitation. Other propagation methods include weak password brute force attacks, SQL attacks, P2P file-sharing attacks, and email-based social engineering attacks, such as the ILOVEYOU and Melissa worms. An important quantitative measure is the basic reproduction number R_0 . If the value is greater than 1, the propagation is said to take place. When the value is less than 1, the propagation will eventually die out.

After the host is infected with the worm, the payload is executed. It can include attacks such as bandwidth consumption to carry out Denial of Service (DoS), keylogging, ransomware distribution, and botnet creation. The payload of the Stuxnet worm targeted the control systems used to control the operation of the centrifuges. Before the payload is executed, the worm creates its own persistence using techniques such as registry key modification, scheduled task creation, and process injection.

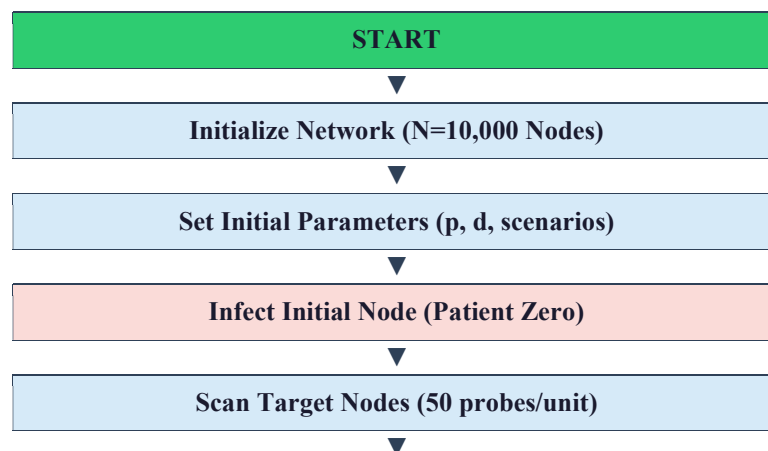
3.3. Methodology Flow

The simulation methodology used in this research study utilizes a stepwise procedure. Each stage involved from network initialization to the analysis of results is clearly specified to increase consistency and facilitate a systematic evaluation. The procedure used in this research is as follows:

1. Start – The simulation begins with all parameters set to default levels.
2. Initialize Network – The generation of scale-free networks consisting of $N = 10,000$ nodes is performed with a relatively smaller number of hub nodes having high degrees of connectivity while the majority of other nodes have low degrees of connectivity which mimics Internet topology.
3. Set Initial Parameters – The parameters for simulation are set with an initial value of $p = 0.3$ as the probability of infection, $d = 0.4$ as the probability of detection, and the number

of probe packets sent out per second is set at 50. Three scenarios, namely Baseline, Patching, and Active Detection, are also created.

4. Infect Initial Node – An initial node infected with worm (patient zero) triggers infection.
5. Scan Target Nodes – At every time step, infected nodes send out 50 probe packets to random IP targets within the network.
6. Propagate Worm – The node becomes infected with a probability of p if any of the probe packets successfully infect a susceptible node.
7. Apply Detection – If the active detection scenario is implemented, each infected node will be detected and isolated by the IDS with probability d .
8. Apply Patching – Infected nodes which are detected become immune and become part of the Recovered class of nodes. Update SIR Counters — The counts of Susceptible (S), Infected (I), and Recovered (R) nodes are updated at the end of each time step.
9. Termination Test – If the simulation has attained its maximum iteration count of $T = 100$ or if there are no more infected nodes left, then the program will terminate; else, the control is transferred back to Step 5.
10. Data Collection and Output – The infection curves for all the three cases are recorded and analyzed.
11. Conclusion – The simulation ends here.



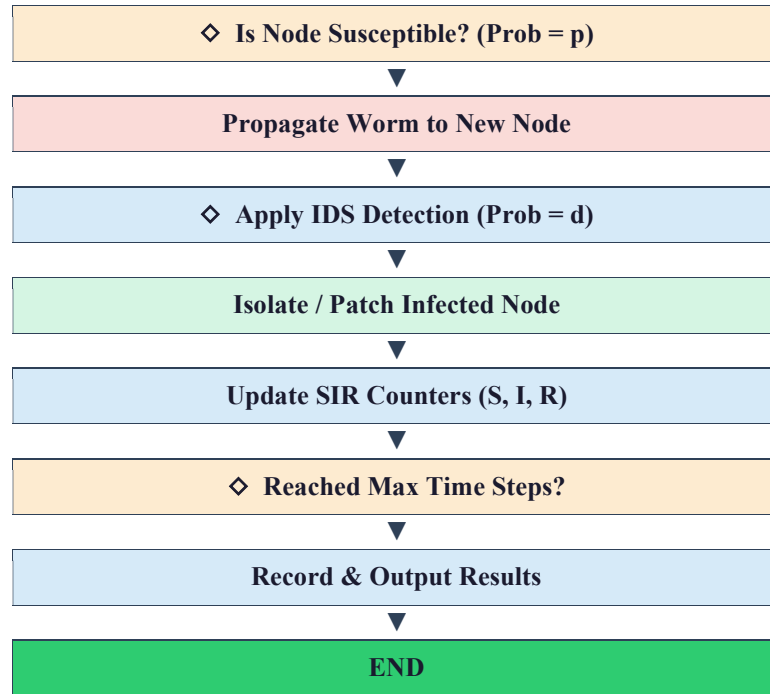


Figure 1. Methodology Flow Diagram for Worm Propagation Simulation

3.4. Algorithm for Worm Propagation Simulation

The algorithm that has been provided below is actually an algorithmic presentation of the process

that was described through the methodology flow. This algorithm can be converted into either a flowchart or a block diagram. The lines have been numbered for reference purposes.

1	INPUT: N = 10000 {total network nodes}
2	p = 0.3 {infection probability per probe}
3	d = 0.4 {detection probability per time unit}
4	T = 100 {maximum simulation time steps}
5	OUTPUT: I[t] {infection count at each time step t}
6	_____
7	BEGIN
8	Initialize network with N nodes
9	Mark all nodes as Susceptible (S)
10	Infect node[0] → set as Infected (I)
11	FOR t = 1 TO T DO

12	FOR each Infected node i DO
13	Generate 50 random target addresses
14	FOR each target node j DO
15	IF node[j] is Susceptible THEN
16	Generate random $r \in [0, 1]$
17	IF $r < p$ THEN
18	node[j] $\leftarrow$ Infected
19	END IF
20	END IF
21	END FOR {end target loop}
22	END FOR {end infected node loop}
23	FOR each Infected node i DO
24	Generate random $q \in [0, 1]$
25	IF $q < d$ THEN {IDS detects worm}
26	Isolate node[i]
27	Apply patch to node[i]
28	node[i] $\leftarrow$ Recovered (R)
29	END IF
30	END FOR {end detection loop}
31	I[t] $\leftarrow$ COUNT of Infected nodes
32	Store I[t], S[t], R[t] for analysis
33	END FOR {end time step loop}
34	OUTPUT I[t] for all t
35	END

Algorithm 1: Worm Propagation and Detection Simulation (SIR Model)

The algorithm operates in three distinct steps: (1) setting up the initial conditions for the network status, (2) conducting a step-by-step simulation involving infection, detection, and patching, and (3) gathering the output results. The modular design makes it possible for each step to be evaluated and customized separately, thus allowing the algorithm to fit different networks, worms, or defenses.

4. EXPERIMENTAL

4.1. Simulation Setup

To examine propagation dynamics, a theoretical simulation based on a modified SIR model, adapted for a computer network, was designed and implemented. The network under investigation contains 10,000 nodes, with a scale-free network configuration, where a small number of nodes are highly connected, while most nodes are poorly connected, similar to a real-world Internet network configuration. The initial configuration of this network contains a single infected node, with all other nodes susceptible to attack. Each infected node sends

probes to 50 addresses in a network in a single time unit, with a 0.3 probability of infecting a susceptible host with each probe. There are three scenarios under investigation: Baseline — no defenses, Patching — 20% of nodes are pre-immunized, and Active Detection — IDS systems are used to isolate infected nodes with a 0.4 probability per time unit.

4.2. Results and Limitations

In the Baseline scenario, the growth of the infected population was exponential, rising from 1 infected node to 5,000 by the 20th time unit, reaching 90% of the network by the 40th time unit, as predicted by the results of the previous studies. The peak level was reduced to 60% with the implementation of the Patching strategy, with the reduction process starting from the 35th time unit. The Active Detection strategy showed promising results, with the infected population kept below 30%. The results also showed that the speed of the reaction of the IDS is an important factor; otherwise, the worm will outrun the reaction. The assumptions used in the simulation are simplified, with approximate values used; the real networks are much more complicated, but the trends observed are in accordance with the known theoretical models.

5. RESULTS AND DISCUSSIONS

5.1. Impact of Worm Attacks

Worm incursions have several damage levels. The network level is affected by scanning activities that cause a Denial-of-Service-like bandwidth congestion before payload execution. The system level is affected by high CPU and memory usage, which slows down systems. The WannaCry ransomworm, which struck in 2017 and made use of the EternalBlue vulnerability in Windows SMB, hit the United Kingdom's National Health Service, causing cancelled operations and diverted patients. The damage level in such scenarios is life-threatening.

5.2. Detection Challenges and Comparison of Methods

The most widely used detection technique is signature-based detection, which has been found to effectively counter known worms. However, it has shown limited efficacy in dealing with new or modified worms. Zero-day worms use unknown weaknesses in a network. They spread before signatures for these unknown threats are ever developed. Anomaly-based detection monitors changes in known network behavior, helping to detect unknown worms. However, this technique often generates false positives, which can prove to be a major problem. Behavior-based detection examines the actions of processes instead of code. It has more powerful analysis capabilities, though it requires more computational resources.

Another new trend in worm attacks is the use of encrypted worm traffic. With the increase in the use of Transport Layer Security (TLS), deep packet inspection (DPI) techniques have shown less efficacy in counteracting such threats. Machine learning techniques, which use metadata instead of decrypting the data, have shown promise in helping to counter such threats. A multi-layered approach that incorporates all three detection techniques, along with segmentation and least privilege access control, has shown the most promise in helping to counter the threat of worms.

CONCLUSION

This paper attempts to assess the worm attacks in the domain of cyber security, specifically in terms of the architecture of the worms, the mechanism of propagation, the history of the worms, and the detection of these worms. The simulation of the SIR model has shown that even partial patching can help in reducing the spread of the infection. The history of worms, from the Morris worm to Conficker, has shown the evolution of worms over the last few decades. Detection of these worms is a major

challenge, specifically in terms of zero-day attacks and encrypted communications. The most effective solution for these attacks is the implementation of a multi-layer security solution. The aim of this work is to help students, researchers, and network administrators in understanding these attacks more effectively.

LIST OF ABBREVIATIONS

C&C: Command and Control; DDoS: Distributed Denial of Service; DPI: Deep Packet Inspection; IDS: Intrusion Detection System; IP: Internet Protocol; P2P: Peer-to-Peer; RPC: Remote Procedure Call; SIR: Susceptible-Infected-Recovered; SMB: Server Message Block; TLS: Transport Layer Security; VLAN: Virtual Local Area Network.

REFERENCES

- [1] C. C. Zou, W. Gong, and D. Towsley, "Code Red worm propagation modeling and analysis," in *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS '02)*, Washington, D.C., USA, Nov. 2002, pp. 138–147.
- [2] D. Moore, C. Shannon, and K. Claffy, "Code-Red: A case study on the spread and victims of an Internet worm," in *Proceedings of the 2nd ACM SIGCOMM Internet Measurement Workshop*, Marseille, France, 2002, pp. 273–284.
- [3] P. Porras, H. Saidi, and V. Yegneswaran, "An analysis of Conficker's logic and rendezvous points," SRI International Technical Report, Feb. 2009. [Online]. Available: <https://mtc.sri.com/Conficker/>
- [4] R. Kaur and M. Singh, "A survey on zero-day polymorphic worm detection techniques," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 3, pp. 1679–1707, Third Quarter 2014.
- [5] F. Manavi and A. Hamzeh, "A novel worm detection approach based on SVM and traffic behavior analysis," in *Proceedings of the IEEE 4th International Conference on Knowledge-Based Engineering and Innovation (KBEI)*, Tehran, Iran, Dec. 2017, pp. 0752–0757.
- [6] E. H. Spafford, "The internet worm program: An analysis," *ACM SIGCOMM Computer Communication Review*, vol. 19, no. 1, pp. 17–57, Jan. 1989.
- [7] Microsoft Security Response Center (MSRC), "MS08-067: Vulnerability in Server Service could allow remote code execution," Microsoft Security Bulletin, Oct. 2008. [Online]. Available: <https://docs.microsoft.com/en-us/security-updates/securitybulletins/2008/ms08-067>
- [8] N. Falliere, L. O. Murchu, and E. Chien, "W32.Stuxnet Dossier," Symantec Security Response, Version 1.4, Feb. 2011. [Online]. Available: https://www.symantec.com/content/en/us/enterprise/media/security_response/whitepapers/w32_stuxnet_dossier.pdf
- [9] MalwareTech Blog, "How to Accidentally Stop a Global Cyber Attack," May 2017. [Online]. Available: <https://www.malwaretech.com/2017/05/how-to-accidentally-stop-a-global-cyber-attacks.html>
- [10] W. O. Kermack and A. G. McKendrick, "Contributions to the mathematical theory of epidemics," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 115, no. 772, pp. 700–721, Aug. 1927.